

테스트 계획 및 결과 보고서

오한빈(팀장) — 통합·작성 · 내용 기여: 각 문서 「기여 이력」 · 검토: 팀 전원

SKN 4차 단위 프로젝트 · 팀 save the pet · 제품 PetTriage

차 례

1

13

테스트 계획 — PetTriage (4차)

2

1.1

0.

4차의 테스트는 3차와 성격이 다르다

2

1.2

1.

다섯 층

2

1.3

2.

지금 무엇이 도는가 — 실측

3

1.4

3.

① 단위 603건 — 승계

10

1.5

4.

② 통합 63건 — **그중 35건 재작성**

11

1.6

5.

③ Django 앱 — **▲** 지금 0건이다

12

1.7

6.

④ 회귀 확인 (8단계) — 4차 발표의 핵심 카드

13

1.8

7.

CI 게이트

14

1.9

8.

무엇을 테스트하지 않는가

15

1.10

9.

선행 관계

15

1.11

10.

미결

16

1.12

11.

기여 이력

16

1.13

12.

검수 기준

16

표기 — 원본 문서는 이모지로 상태를 나타낸다. 인쇄용 PDF에서는 폰트에 있는 기호로 바꿨다: ● 완료·검증됨 ·
● 구현됐으나 검증 안 됨 · ▲ 미구현·주의가 필요함 · □ 미착수 · ※ 주의. 뜻은 각 문서의 범례와 같다.

제 1 장

13 · 테스트 계획 — PetTriage (4차)

SKN 4차 단위 프로젝트 · 팀 save the pet · 제품 PetTriage **통합: 오한빈(팀장)** · 내용 기여: § 11 기여 이력 · 검토: 전원 상위 문서: 10_요구사항정의서.md(무엇을) · 11_화면설계서.md(어디서) · 14_전환설계.md(왜)
최초 작성: 2026-08-24 · 갱신 **2026-08-27** · pytest **666 passed** · **22 deselected**

1.1 0. 4차의 테스트는 3차와 성격이 다르다

	3차	4차
물음 수단 문서	“RAG 가 얼마나 잘 답하나” 골든셋 60건 · 과소평가율 · 근거 검증 04_테스트-평가계획.md (archive)	“전환해도 같은 답이 나오나” 단위 승계 + 통합 재작성 + 회귀 확인 이 문서

3차의 품질 평가는 끝났고 그 결과는 docs/archive-3rd/ 에 있다. **4차는 기능 테스트다.** 다만 골든셋 60건은 버리지 않고 **회귀 판정 도구**로 쓴다 (§ 6).

▲ 4차의 핵심 주장은 “판정이 바뀌지 않았다” 이다 (D-102). 이 문서의 절반은 그 문장을 증명하는 절차다.

▲ 이것이 테스트 계획·결과의 정본이다. 제출용 .xlsx 는 여기서 생성한다.

```
make testplan-xlsx # macOS · Linux
python scripts/build_testplan_xlsx.py --write # Windows (make 가 없다)
```

사람이 고치는 파일은 이 .md 하나뿐이다. 팀원 원본(docs/ksr/...xlsx)은 기여 근거로 남고, 거기서 정본으로 옮겨 적는 일은 § 2.2 에서 한 번만 일어난다. 요구사항정의서와 같은 규약이다 (10 § 0) — 사본을 두면 한쪽만 고쳐진다 (D-22).

1.2 1. 다섯 층

- ① 단위 603건 프레임워크 무관 — 그대로 승계 § 3
- ② 통합 63건 FastAPI 결합 — 35건만 재작성 § 4
- ③ Django 0건 ▲ Django 앱 코드가 검사되지 않는다 § 5
- ④ 회귀 60건 골든셋 — 전환 전후 동일한가 § 6
- ⑤ 수동 UI 51건 [신규] 사람이 브라우저로 확인한다 § 2.1

①②의 합이 현재 pytest 가 세는 **666**다. ③은 아직 없고, ④는 pytest 밖에서 돈다.

[신규] ⑤가 **2026-08-27 에 늘었다.** 권소라가 화면 51건을 손으로 돌렸다 (docs/ksr/...(최종).xlsx).

처음엔 이것을 ③이 0건인 동안의 임시방편으로 적었다. 그 판단이 틀렸다. 같은 날 ⑤의 비고 칸에서만 결함이 셋 나왔고, 그때 ①②는 657건이 전부 초록이었다.

비고 한 줄	실제로 무엇이었나
“성장기 제외 조건은 미검증” “포도 답해도 재차 되질문 후 응답불가” (시연 로그)	화면과 서버가 반대 답을 냈다 (D-108) 실제 엔진이 앞 턴을 잃었다 (D-110 · UC-04) 다이어리를 여는 것만으로 LLM 호출 이 나갔다 (D-109)

⑤는 ①②가 구조적으로 못 보는 것을 본다. 자동화는 “되는가”를 묻고, 사람은 “이상하지 않은가”를 묻는다. 위 셋은 전부 **아무 데도 안 틀린 채로** 이상했다 — 화면 정상, 테스트 초록, 요구사항 충족.

▲ 그래도 ⑤는 ③을 대신하지 못한다. 사람이 돌린 것은 **회귀를 못 잡는다**. 위 셋이 이제 안 돌아오는 것은 ⑤ 덕분에 아니라 **찾은 것을 ①로 옮겨 적었기 때문이다** (test_lifestage.py 30건 · test_clarify_multiturn.py 9건).

⑤는 **발견하고 ①이 지킨다**. 둘 중 하나만 있으면 안 된다 — ⑤만 있으면 다음 사람이 되돌려도 모르고, ①만 있으면 애초에 못 찾는다.

1.3 2. 지금 무엇이 도는가 — 실측

pytest
666 passed, 22 deselected in 26.92s

값	수	근거
수집·실행 제외 (todo·gpu)	666 22	pytest (2026-08-27) addopts = -m 'not todo and not gpu'
테스트 함수	499	parametrize 가 666건으로 펼친다. 측정: <code>grep -c '^s*def test_' tests/*.py</code>
FastAPI 결합	63	test_api 28 · test_auth_api 28 · test_records_api 7
프레임워크 무관	603	666 - 63

세는 단위를 섞지 않는다. 위 수는 전부 **pytest 수집 기준**이지 def test_ 개수가 아니다. 옛 문서가 460개 중 60개(13%) 라고 적었는데, 분자는 함수 수이고 분모는 출처 불명이라 **단위가 어긋난 비율**이었다 (14 § 1.1 각주).

다시 재려면 — pytest 로 전체를, pytest <파일들> --collect-only -q 로 부분을 센다. 후자는 addopts 의 -q 와 겹쳐 -qq 가 되어 **합계 줄이 사라지니 파일별 수를 더한다**.

1.3.1 2.1 ⑤ 수동 UI 51건 — 무엇이 실제로 만져졌나

출처: docs/ksr/PetTriage_테스트계획및결과보고서(최종).xlsx (권소라 · 2026-08-26~27). 화면 ID 는 그쪽 체계(SC-001~008)를 쓴다 — 정본 대응표는 11 § 9.1.

상태	수	뜻
통과	34	브라우저·서버에서 재현했고 관측값이 적혀 있다
참고필요	13	자동화 테스트는 있으나 실제 서비스 경로가 아니다
미실행	4	아직 아무도 안 돌렸다

[갱신] 2026-08-27 갱신 — 미실행 19건 → 4건. 첫 판(...(UI).xlsx)에서 비어 있던 인프라 15건(TC-NFR-*)을 실제 배포에 대고 다 썼다. 배포가 끝나 있었기에 가능했던 일이고 (12 § 6), 그 덕에 10 § 6의 □ 여섯 개가 실측으로 채워졌다. 두 판이 다 남아 있어 무엇이 어떻게 바뀌었는지 셀 수 있었다 — .xlsx 로는 드문 행운이다.

결과 칸이 서술이 아니라 관측값이라 좋은 보고서다. 예를 들어 체중 급변(TC-FR-DIARY-010)은 경계 세 점을 실제로 넣었다 — 4.20→4.30(+2.38%) 미발생 → 4.45(+5.95%) 주의 → 4.65(+10.71%) 병원. DIARY_WEIGHT_ALERT_*_PCT 의 5 · 10 과 맞는다.

1.3.1.1 ▲ “참고필요” 12건이 7b 를 가리킨다

핵심 리스크 2 — “test_auth_api.py · test_records_api.py 는 FastAPI 자체 인증/기록 라우터를 테스트하는데, 이 라우터는 실제 서비스에서 쓰이지 않는(Django 로 대체된) 병행 구현으로 보임 — 통과해도 실제 화면 동작을 보증하지 않음”

우리가 D-104 ④ 로 정리한 것과 같은 결론에, 코드 대조만으로 따로 도달했다. ②의 63건이 왜 재작성 대상인지를 이 12건이 바깥에서 증명한다 (§ 4).

1.3.1.2 이 보고서가 잡은 실제 버그 — Pet 의 정렬

TC-FR-CHAT-006 비고: “Pet.Meta 에 ordering 이 없어 .first() 가 pk(랜덤 UUID) 순으로 동작. 현재는 등록순과 우연히 일치하나 신규 등록 시 뒤집힐 수 있음”.

확인 결과 사실이였다. .first() 에 기대는 화면이 셋이다 — chat/views.py(펫 없이 /chat/ 진입) · chat/context_processors.py(사이드바) · diary/views.py(기록장 기본 펫). ordering = ["created_at"] 로 못 박았다 (2026-08-27).

자동화가 못 잡는 종류다. 테스트를 돌려도 초록이고, 펫이 하나뿐이면 영원히 안 드러난다. “우연히 맞는다” 를 알아보는 것은 지금으로선 사람뿐이다 — ⑤를 두는 이유가 이것이다.

1.3.1.3 이 보고서와 우리 문서가 어긋난 자리

TC-FR-HIST-001~004 가 통과다. 우리 11 § 4.5 는 503 이라고 단언했었다. 우리 쪽이 틀렸다 — 환경 의존적인 사실을 절대적으로 적었다. 11 § 4.5 정정 참조.

1.3.1.4 ▲ 최종본에서 한 줄은 받지 않았다 — TC-NFR-PERF-001

상태는 통과인데 실제결과는 “현재 적용하지 않았음 … 검증 대상에서 제외함” 이다. 안 했고 안 했다고 적어 놓고 통과다. 그리고 근거로 든 “정적 파일 참조가 없다” 도 django.contrib.admin(= 우리 명세의 SC-07)을 빠뜨렸다. 판정 근거는 10 § 6.1.

이 한 줄이 나머지 33건의 값어치를 깎지 않는다. 오히려 반대다 — 34건이 관측값과 함께 적혀 있어서 이 한 줄만 결이 다른 것이 눈에 띄었다.

1.3.1.5 ⑤가 아직 못 채운 칸

- 미실행 4건 — TC-FR-DIARY-009(조류 배설물) · TC-NFR-DEPLOY-001(EC2 배포) · TC-NFR-DEPLOY-004(Docker Compose) · TC-NFR-DEPLOY-006(DB 백업)
- ▲ TC-NFR-DEPLOY-001 이 “기능 미구현” 인데 같은 시트의 002 · 003 · 005 는 실제 EC2 에서 확인한 내용이다 (“EC2 파일 권한 문제로 사진이 처음엔 안 보였으나…”). 요구사항 문구가 “EC2 배포”와 “Docker Compose”를 한 줄에 묶어 생긴 혼란이고, D-107 이 가른 바로 그 지점이다. 10 에서는 NFR-20(systemd)과 분리해 두었다
- ▲ TC-NFR-DB-001(“동일한 인스턴스로 적용” · 통과)은 D-104 와 충돌한다 — 우리는 웹앱 DB 와 3차 DB 를 갈랐는데 배포는 하나로 합쳤다. 채팅 내역이 보이는 이유가 그것이고(11 § 4.5), 동시에 D-104 가 그은 경계가 배포에서 무너져 있다는 뜻이다. 7b 입력
- FR-CHAT-007(표기 교정 안내)에 대응하는 TC 가 없다 — 요구사항표에는 있는데 테스트 목록에 빠졌다. G-042 와 직접 닿는 항목이라 (§ 6.4) 채워야 한다 (10 의 FR-39)

1.3.1.6 ▲ 참고필요 13건은 두 가지 다른 뜻이다

합칠 때 이 둘을 갈라야 한다 — 대응이 정반대다.

뜻	수	무엇을 해야 하나
(A) 자동화가 엉뚱한 코드를 검사한다 — 병행 라우터(§ 4)	10	7b 에서 테스트를 다시 쓴다
(B) 사람이 돌렸으나 일부 조건이 미검 증	3	그 조건을 확인한다

(B) 셋은 TC-FR-CHAT-005 · TC-FR-CHAT-006 · TC-FR-DIARY-010 이고, **그중 둘에서 실제 결함이 나왔다** (D-110 · D-108). 남은 하나(CHAT-006 정렬)도 같은 날 고쳐졌다 (Pet.Meta.ordering).

(B) 는 “거의 통과” 가 아니라 “여기를 더 보라” 는 표시였다. 셋 중 셋이 그랬다. 다음 판에서는 (B) 를 **미검증 조건 목록**으로 따로 세운다.

● **TC-FR-DIARY-011(스트릭)**은 **최종본에서 통과로 바뀌었다** — ” 연속 기록 2일째” 카드 확인. 첫 판의 “추후 개발 예정” 은 사유가 틀렸던 것이 맞았다.

1.3.2 2.2 수동 테스트케이스 51건 — 결과

▲ ID 는 **정본 것으로 옮겨 적었다**. 원본은 FR-AUTH-002 처럼 분야 접두어를 쓰고 이 문서·10·추적표는 FR-03 처럼 평면 번호를 쓴다. 대응 규칙은 10 § 11.

전문은 원본에 있다 — 아래 관측 칸은 값만 남긴 것이다. 출처: docs/ksr/PetTriage_테스트계획및결과보고서(최종).xlsx (권소라 · 2026-08-26~27).

TC	요구사항	상태	관측
AUTH-001	FR-01 · FR-02 · FR-27	※ 참고필요	자동화는 있으나 FastAPI 병행 라우터 대상 (tests/test_auth_api.py::TestSignup)
AUTH-002	FR-03	※ 참고필요	자동화는 있으나 FastAPI 병행 라우터 대상 (tests/test_auth_api.py::TestLogin)
AUTH-003	FR-04	● 통과	테스트 통과 user02 이용 중 로그아웃 시 로그인 화면으로 이동 F12 -> Network 기록 삭제 확인
AUTH-004	FR-28	※ 참고필요	자동화는 있으나 FastAPI 병행 라우터 대상 (tests/test_auth_api.py)
AUTH-005	FR-29	● 통과	curl 미인증 요청 → HTTP 302, Location: /accounts/login/?next=/pets/ 확인 (Server: WSGIServer). 브라우저에서도 로그인 화면 표시됨 · @login_required
PET-001	FR-06 · FR-08	※ 참고필요	자동화는 있으나 FastAPI 병행 라우터 대상 (tests/test_auth_api.py::TestPets)
PET-002	FR-31	※ 참고필요	자동화는 있으나 FastAPI 병행 라우터 대상 (tests/test_auth_api.py::TestPets)
PET-003	FR-07	● 통과	프로필 설정 진입 정보(이름, 나이,성별,체중,소개,특이사항,품종,크기)/사진등록/사진 삭제/사진수정/ → POST /pets/edit/ 302 → GET /pets/ 200 목록 카드에 반영 확인

TC	요구사항	상태	관측
PET-004	FR-34	※ 참고필요	자동화는 있으나 FastAPI 병행 라우터 대상 (tests/test_auth_api.py::TestPets)
PET-005	FR-33	● 통과	프로필 설정 진입 사진등록/사진삭제/사진수정/ → POST /pets/edit/ 302 → GET /pets/ 200 목록 카드에 반영 확인
PET-006	FR-32	● 통과	한 계정에 2마리 등록확인. 목록에서 A 카드 클릭 → http://127.0.0.1:8080/chat/?pet_id=d9633... 200, 사이드바에 A 표시. B 카드 클릭 → 다른 pet_id로 200, 사이드바에 B 표시. 주소창에 pet_id 직접 입력해도 ...
CHAT-001	FR-36	● 통과	/chat/?pet_id=bb687f... 진입 시 사이드바에 (소형견·2살) 표시, 헤더 문구에 이름 반영됨. 입력창 위 예시 chip N개 표시(강아지3개, 고양이2개,앵무새1개) 클릭 시 입력창에 채워짐. 다른 펫으로 전환 시 이름·정보 함께 변경
CHAT-002	FR-10 ~ FR-14	● 통과	자동화 테스트 통과 — tests/test_api.py, tests/test_triage_gate.py, tests/test_grounding.py, tests/test_triage_evidence.py (2026-08-26, pytest 582 passed) · FastAPI /api/ask
CHAT-003	FR-15	● 통과	자동화 테스트 통과 — tests/test_slot_schema.py, tests/test_state_reaches_response.py (2026-08-26, pytest 582 passed)
CHAT-004	FR-16	● 통과	자동화 테스트 통과 — tests/test_unknown_substance.py, tests/test_route1.py (2026-08-26, pytest 582 passed)
CHAT-005	FR-37	※ 참고필요	동일 펫 내 연속 질문 시 session_id 동일 유지 확인 (F12 Network 요청 payload 2회 비교). 펫 전환 시 새 session_id 발급, 다시 원래 펫으로 복귀해도 이전 세션과 다른 id 생성 — 펫 전환이 세션 경계로 동작함. · 되묻기 기반 맥락 인계는 미검증 — 응급도 높은 소재는 즉시 판정(되묻기 생략) 0알 먹었어 -> 되질문 발생 -> 포도

TC	요구사항	상태	관측
CHAT-006	FR-38	※ 참고필요	pet_id 없이 /chat/ 접속 → 200, 사이드바에 “냥냥” 표시. shell 확인 결과 등록순 첫 번째(냥냥)와 일치 상태: 통과 · Pet.Meta에 ordering이 없어 .first()가 pk(랜덤 UUID) 순으로 동작. 현재는 등록순과 우연히 일치하나 채팅 내역 진입 시 세션 카드 7건 표시, 최신순(내림차순) 정렬 확인. 화면 표시 순서 09:29 → 09:28 → 09:27 → 09:19 → 09:17 → 09:17 → 09:16, DB ChatSession.objects.order_by('-created_at'...
HIST-001	FR-23	● 통과	user01 세션 URL(/history/01a3205860.../)을 user02 로그인 상태에서 직접 입력 → HTTP 404 확인(F12 Network 탭). user02의 채팅 내역 목록에 user01 세션 미노출 확인. · 보안
HIST-002	FR-26	● 통과	한 세션에서 “당근 줘도 돼?” → “초콜릿 먹었어” 2회 질의 후 상세 진입. 질문 2건·답변 2건이 발화 순서대로 표시되고 질문·답변 짝이 일치함을 확인. 채팅 화면 내용과 상세 화면 내용 동일.
HIST-003	FR-25	● 통과	위 세션(관찰성 질의 + 응급 질의 존재)의 목록 카드 배지가 “지금 연락”으로 표시됨. 카드 미리보기 문구는 첫 질문이나 배지는 두 번째 응답의 응급도를 반영 — 세션 내 최고 응급도 표시 동작 확인.
HIST-004	FR-24	● 통과	자동화는 있으나 FastAPI 병행 라우터 대상 (tests/test_records_api.py)
DIARY-001	FR-09	※ 참고필요	자동화는 있으나 FastAPI 병행 라우터 대상 (tests/test_records_api.py)
DIARY-002	FR-40	※ 참고필요	자동화는 있으나 FastAPI 병행 라우터 대상 (tests/test_records_api.py)
DIARY-003	FR-41	※ 참고필요	자동화는 있으나 FastAPI 병행 라우터 대상 (tests/test_records_api.py)
DIARY-004	FR-42	● 통과	3개 날짜에 체중 3.0/3.0/3.3 저장 → 좌측 패널에 SVG 라인 차트 표시, 점 3개·연결선 확인(F12 Elements에서 path.circle 3개 확인). 현재 체중 3.3kg, 증감 +0.3kg 표시

TC	요구사항	상태	관측
DIARY-005	FR-43	● 통과	소형견 기준(범위 2~7kg) 1.9kg → “저체중 가능성” / 4.2kg → “정상 체중 범위” / 7.5kg → “과체중 가능성” 표 시 확인.
DIARY-006	FR-40	● 통과	/diary/ 진입 시 2026년 8월 캘린더 표시. 기록 저장한 8/24·8/26 날짜에 초록 표시 확인, 기록 없는 날은 표시 없 음. <>로 7월·9월 이동 시 해 당 월 기록 표시가 함께 갱신 됨. 날짜 클릭 시 우측에 해당 일자 기록 표시·체중변화 그 래프는 누적으로 월별로 나뉘 어지지 않음
DIARY-007	FR-20	※ 참고필요	자동화는 있으나 FastAPI 병 행 라우터 대상 (tests/ test_records_api.py)
DIARY-008	FR-20	● 통과	/diary/ → 「기록 리포트 다 다운로드」 최근 1개월 선택 후 다운로드. Django 로그에 POST /internal/report/ summarize HTTP/1.1 200 OK 확인, FastAPI 요약 위임 동작. 생성된 txt 파일에 요약 문단 포함 — 실제 기…· FastAPI 내부 API
DIARY-009	FR-46	□ 미실행	미실행 — 수동 테스트 필요· 추후에 추가 예정
DIARY-010	FR-44	※ 참고필요	임계값 경계 3케이스 확인. 4.20→4.30(+2.38%) 알림 미발생 / 4.20→4.45(+5.95%) “당분 간 지켜보세요” / 4.20→4.65(+10.71%) “병 원 방문 권장” 표시 확인. F12 Network 탭에서 GET /api/ report 응답 wei…·GET / api/report weight_alert 필드 성장기(새끼) 제외 조건 은 미검증 — 추가 확인 필요
DIARY-011	FR-45	● 통과	/diary/ 좌측 패널에 ” 연속 기록 2일째” 카드 표시 확인. 8/26·8/27 연속 기록 상태와 일치. 캘린더의 기록 표시일 과 대조 확인.·GET /api/ report streak 필드
UI-001	11 § 2 화면 규칙	● 통과	반려동물 선택을 제외한 모든 메뉴는 동일한 사이드바를 제 공
UI-002	11 § 2.1	● 통과	반려동물 정보 무게나 사진이 정상적으로 출력·context processor
UI-003	NFR-18	● 통과	반응형으로 다른 사이트와 같 이 햄버거 버튼이 사용되어 정상 작동

TC	요구사항	상태	관측
UI-004 N-ARCH-001	11 § 2 화면 규칙 NFR-13	● 통과 ● 통과	메뉴 활성화 정상작동 확인 시에는 FastAPI로 Django는 나머지를 구현 문제 없이 프록시적용 되어있 음 · Docker 동일한 인스턴스로 적용 JWT 토큰 방식에서 세션으로 변경하여 적용 완료 Django CsrfViewMiddleware를 활 성화하고, 로그인·회원가입 폼에는 CSRF 토큰을 삽입함. 세션 기반 DRF POST API(/ api/records)는 CSRF 쿠키 와 X-CSRFToken 헤더를 함 께 전송하도록 구현했으며, HTTPS 환경에서 로그인… 실제 다른 링크로 들어갔을 시 Not Found Exception이 발생하여 접근이 불가능함 회원가입 시 Django User.objects.create_user() 를 사용하여 비밀번호를 저장 함. Django 기본 PBKDF2-SHA256 해싱이 적 용되며, DB에는 평문 비밀번호가 아닌 솔트가 포함된 해 시값만 저장됨. 안 했다고 적고 통과로 표시 됐다 — 10 § 6.1 기능 미구현 추후에 추가할 예정 · RDS 미사용 추후에 추 가 예정 실제 사이트는 HTTPS로 EC2 인스턴스 내부에서는 HTTP로 되어 있음 · Let's Encrypt 미사용 기준은 도커 기준이지만 기능 적으로는 문제 없이 잘 작동 기능 미구현 추후에 추가할 예정 · 추후에 추가 예정 EC2 파일 권한 문제로 반려동 물 사진이 처음엔 안보였으나 권한 부여 후 문제없이 저장되 었고 표기까지 됨 · S3 미사용 기능 미구현 추후에 추가할 예정 · 추후에 추가 예정 env 기반으로 환경 설정 완료
N-ARCH-002	NFR-14	● 통과	
N-DB-001 N-SEC-001	▲ D-104 충돌 NFR-15	● 통과 ● 통과	
N-SEC-002	NFR-16	● 통과	
N-SEC-003	FR-07 · FR-26	● 통과	실제 다른 링크로 들어갔을 시 Not Found Exception이 발생하여 접근이 불가능함 회원가입 시 Django User.objects.create_user() 를 사용하여 비밀번호를 저장 함. Django 기본 PBKDF2-SHA256 해싱이 적 용되며, DB에는 평문 비밀번호가 아닌 솔트가 포함된 해 시값만 저장됨. 안 했다고 적고 통과로 표시 됐다 — 10 § 6.1 기능 미구현 추후에 추가할 예정 · RDS 미사용 추후에 추 가 예정 실제 사이트는 HTTPS로 EC2 인스턴스 내부에서는 HTTP로 되어 있음 · Let's Encrypt 미사용 기준은 도커 기준이지만 기능 적으로는 문제 없이 잘 작동 기능 미구현 추후에 추가할 예정 · 추후에 추가 예정 EC2 파일 권한 문제로 반려동 물 사진이 처음엔 안보였으나 권한 부여 후 문제없이 저장되 었고 표기까지 됨 · S3 미사용 기능 미구현 추후에 추가할 예정 · 추후에 추가 예정 env 기반으로 환경 설정 완료
N-SEC-004	FR-02	● 통과	
N-PERF-001	NFR-17	▲ 미적용	
N-DEPLOY-001	NFR-20	□ 미실행	
N-DEPLOY-002	NFR-22	● 통과	실제 사이트는 HTTPS로 EC2 인스턴스 내부에서는 HTTP로 되어 있음 · Let's Encrypt 미사용 기준은 도커 기준이지만 기능 적으로는 문제 없이 잘 작동 기능 미구현 추후에 추가할 예정 · 추후에 추가 예정 EC2 파일 권한 문제로 반려동 물 사진이 처음엔 안보였으나 권한 부여 후 문제없이 저장되 었고 표기까지 됨 · S3 미사용 기능 미구현 추후에 추가할 예정 · 추후에 추가 예정 env 기반으로 환경 설정 완료
N-DEPLOY-003	NFR-21	● 통과	
N-DEPLOY-004	▲ 철회 (D-107)	□ 미실행	
N-DEPLOY-005	NFR-21	● 통과	
N-DEPLOY-006	NFR-23	□ 미실행	실제 사이트는 HTTPS로 EC2 인스턴스 내부에서는 HTTP로 되어 있음 · Let's Encrypt 미사용 기준은 도커 기준이지만 기능 적으로는 문제 없이 잘 작동 기능 미구현 추후에 추가할 예정 · 추후에 추가 예정 EC2 파일 권한 문제로 반려동 물 사진이 처음엔 안보였으나 권한 부여 후 문제없이 저장되 었고 표기까지 됨 · S3 미사용 기능 미구현 추후에 추가할 예정 · 추후에 추가 예정 env 기반으로 환경 설정 완료
N-ENV-001	NFR-19	● 통과	

TC 이름에서 TC-FR- · TC-NFR- 접두어를 뺐다 (N- 는 비기능).

1.3.2.1 ▲ 위 표의 수가 § 2.1 과 하나 다르다

	통과	참고필요	미실행	미적용
§ 2.1 — 권소라가 적은 대로	34	13	4	—
§ 2.2 — 검토 후	33	13	4	1

차이는 N-PERF-001(정적 파일 nginx 서빙) 하나다. 원본은 **통과**로 적었는데 실제결과는 “현재 적용하지 않았음 … 검증 대상에서 제외함” 이다. **안 한 것을 통과로 받지 않는다** (D-58). 판정 근거는 10 § 6.1.

두 수를 다 남긴다. 하나로 댔으면 누가 무엇을 보고했는지 와 팀이 무엇으로 받았는지 가 섞인다. 원본은 원본대로 맞고, 이 문서는 검토 결과다.

1.3.2.2 이 표가 요구사항을 얼마나 댔나

	수
⑤가 닿은 FR	42 / 47
⑤가 못 닿은 FR	5

못 닿은 FR	왜
FR-05 내 프로필	▲ 화면이 없다 (11 § 1.2) — 만들지 철회할지 미결
FR-35 사진 관문	①이 본다 (test_image_gate.py 11건)
FR-39 표기 교정	①이 본다 (test_spelling.py · G-042)
FR-30 로그아웃 POST 전용	[신규] 2026-08-27 신설 — 이 표가 만들어진 뒤에 생겼다
FR-47 가입 후 자동 로그인	[신규] 2026-08-27 신설 — 같은 이유

신설 둘은 **코드를 읽다 나온 요구사항**이다. 아무도 그걸 요구한 적이 없는데 코드는 그렇게 동작하고 있었다 — 그래서 화면으로 확인한 사람도 없다. **다음 ⑤ 판에 넣는다.**

측정: § 2.2 표의 요구사항 칸에서 FR-\d\d 를 모아 10 § 5 의 47건과 견췄다.

1.4 3. ① 단위 603건 — 승계

1.4.1 3.1 왜 프레임워크와 무관한가

판정·어휘·검색·생성·설정은 src/pettriage/ 안에 있고 **app/ 을 임포트하지 않는다.** 배달 계층과 도메인을 섞지 않기로 한 D-40 의 결과다.

영역	대표 파일
트리아지 판정 · 게이트	test_triage_gate.py · test_triage_evidence.py · test_basis.py
규칙 · 정량 계산	test_rules.py · test_unknown_substance.py
어휘 · 별칭	test_vocabulary.py (43) · test_aliases.py (29)
검색 · 중복 접기	test_retrieval_filter.py · test_dedupe.py
생성 · 근거 검증	test_verbalize.py (41) · test_grounding.py
그래프 조립	test_graph_build.py · test_state_reaches_response.py
설정 · 모델	test_models_config.py (30) · test_config_is_read.py
안전	test_contacts.py (D-47)

1.4.2 3.2 ● 이미 실측으로 확인됐다

[갱신] 2026-08-27 — 564 → 603. 늘어난 39건은 승계분이 아니라 새로 찾은 결함을 못박은 것이다 (test_lifestage.py 30 · test_clarify_multiturn.py 9 · D-108 · D-110). 셋 다 ⑤의 비고 칸에서 나왔다 (§ 2.1).

Django 3711줄이 들어온 뒤에도 589건이 그대로 통과했다 (2026-08-24 머지 직후 · 당시 수치). 14 § 1.1 ②의 “단위 테스트는 프레임워크와 무관하다”는 이제 주장이 아니라 관측이다.

1.4.3 3.3 승계 판정 기준

전환이 끝난 뒤에도 이 603건은 한 줄도 고치지 않는다. 고쳐야 한다면 그것은 승계가 아니라 도메인이 바뀐 것이므로, 왜 바뀌었는지를 먼저 적는다.

1.5 4. ② 통합 63건 — 그중 35건 재작성

1.5.1 4.1 대응 관계

3차 (FastAPI · JWT)	4차 (Django · 세션)	건수
test_auth_api.py — 회원가입 · 로그인 · 토큰	accounts — 세션 로그인	28
test_api.py — /ask 계약	FastAPI 잔류 — 손대지 않는다	28
test_records_api.py — 기록 · 리포트	diary + 내부 요약 왕복	7

1.5.2 ● 4.1.1 실제 재작성 대상은 35건이다 (2026-08-27 실측)

오래 “63건 재작성”으로 적혀 있었고, 그중 test_api.py 28건을 가르는 것이 미결이었다. 세어 보니 가를 것이 없다.

test_api.py 가 부르는 주소 /api/ask 17 · /api/health 1 ← 그게 전부다
펫 CRUD · 소유권 호출 0건

그리고 conftest.py:29 가 DATABASE_URL 을 비워서 DB 라우터가 아예 등록되지 않은 앱을 테스트한다. test_api.py 는 DB 를 모른다 — 7b 가 지나가도 한 줄도 안 바뀐다.

	건수	7b 에서
test_api.py	28	● 그대로 — /api/ask 는 남는 표면이다
test_auth_api.py	28	▲ 재작성
test_records_api.py	7	▲ 재작성

재작성 35 · 승계 28. 표 머리의 “63건 재작성”은 이제 틀린 말이다 — 63 은 FastAPI 앱에 붙어 있는 수 이지 고쳐야 하는 수가 아니다.

1.5.3 4.2 인증 방식이 바뀌므로 검증 대상도 바뀐다

3차에서 검증하던 것	4차에서 검증할 것
JWT 발급 · 만료 · 서명 검증 Authorization: Bearer 누락 시 401 토큰의 user_id 로 소유권 확인	세션 쿠키 발급 · 만료 · LOGIN_URL 리다이렉트 미인증 접근 시 /accounts/login/ 으로 302 request.user 로 소유권 확인

소유권 검사는 방식이 바뀌어도 반드시 남는다 — “남의 펫이 안 보인다”는 인증 구현과 무관한 요구사항이다 (FR-07 · D-52).

1.5.4 4.3 착수 시점

4·5·6 단계가 끝난 뒤다. FastAPI 의 auth·users·pets·records 라우터를 폐기하는 것과 같은 작업이므로 (검토 § 2 A안), 따로 잡지 않고 묶는다.

1.6 5. ③ Django 앱 — ▲ 지금 0건이다

1.6.1 5.1 무엇이 문제인가

pyproject.toml 의 testpaths = ["tests"] 가 accounts/ · pets/ · diary/ · chat/ · webapp/ 을 보지 않는다. **3711줄 중 CI가 실행하는 것은 한 줄도 없다.** 각 앱의 tests.py 는 3줄짜리 기본 스텝이다.

666건이 초록인 것은 **새 코드를 안 건드려서**다. 초록이 안전을 뜻하지 않는다.

D-48 이 정확히 이 상황이었다. 424줄이 CI 에서 한 줄도 안 돌았고, 테스트를 붙이자마자 passlib/bcrypt 충돌로 **회원가입이 한 건도 안 되는 것**이 드러났다. 시연 당일에 발견됐을 문제였다. 지금은 회원가입·로그인·펫 CRUD·다이어리가 **전부 그 상태**다.

1.6.2 5.2 최소 세트 — 이 넷부터

우선순위 순이다. 위에서부터 하나씩 붙인다.

#	무엇	왜 이 순서	대응
1	마이그레이션이 깨끗한 DB 에서 끝까지 돈다	● 이제 돈다 — 지키는 테스트가 없다	[안전] D-104 완료 판정
2	회원가입 → 로그인 → 세션 유지	D-48 이 여기서 터졌다	UC-01 · UC-02
3	펫 등록 → 목록에 보인다 → 남의 펫은 안 보인다	소유권은 안전 요구사항이다	FR-06 · FR-07
4	기록 작성 → GET /api/report 가 요약물 돌려준다	Django↔FastAPI 왕복 을 검증하는 유일한 자리	FR-09 · FR-20 · 12 § 4.4

4번은 추론 서비스가 떠 있어야 하므로, **뺀다 안고도 도는 형태**로 만든다 — INFERENCE_INTERNAL_URL 로 가는 호출을 대역으로 바꾸고, 실제 왕복은 별도 표시를 단다.

1.6.3 5.3 어떻게 붙이나

Django 테스트는 manage.py test 가 아니라 **pytest 로 모은다** — 게이트를 두 개 만들지 않는다.

```
# pyproject.toml
testpaths = ["tests", "accounts", "pets", "diary", "chat"]
```

pytest-django 를 dev extra 에 넣고 constraints.txt 에 핀을 박는다. **새 의존성은 선언한다** — 이번 머지에서 python-dotenv 가 선언 없이 상시 임포트 경로에 들어갔던 것과 같은 사고를 만들지 않는다 (검토 § 4 · D-70).

1.6.4 ▲ 5.3.1 CI 잡에 django extra 를 함께 넣는다

지금 어느 잡도 **django** 를 깔지 않는다.

```
test      .[api,dev]
db-deps   .[api,db,dev]
rag-deps  .[api,rag,ingest,dev]
```

그래서 Django 쪽은 **테스트가 없을 뿐 아니라 임포트조차 되지 않는다.** 의존성 해결이 깨져도 **사람이 설치할 때까지 아무도 모른다.**

2026-08-26 에 이것이 두 번 드러났다.

무엇	어떻게 드러났나
런처가 django 를 안 깔았다 DRF 3.15.2 가 Django 5.1 을 지원 범위에 안 넣었다	사람이 manage.py 를 치고 ModuleNotFoundError 를 봤다 사람이 메타데이터를 직접 읽어 봤다

둘째는 특히 조용했다 — djangorestframework 3.15.2 의 요구는 django>=4.2 로 **상한이 없어서 pip 가 아무 말도 하지 않는다**. 분류(Framework :: Django :: 5.0 까지)를 읽어야 알 수 있고, 그건 CI 가 볼 수 있는 것이 아니다.

“pip 가 통과시킨다”와 “지원한다고 밝혔다”는 다르다. 3.16 부터 5.1 이 분류에 들어오고, **3.18.0 은 django>=5.2 를 요구해 5.1 을 끊는다.** 그래서 범위를 >=3.16, <3.18 로 고쳤다 (2026-08-26 · 핀 3.16.1).

그리고 그 판올림을 확인할 방법이 사람 손밖에 없었다. 단위 테스트가 DRF 를 한 줄도 안 지나서 다이어리 저장·리포트 조회를 눈으로 봐야 했다. § 5.2 의 최소 4건 중 3·4번이 정확히 그 자리다.

1.6.5 5.4 목표는 개수가 아니다

커버리지 퍼센트를 목표로 걸지 않는다. **§ 5.2 의 넷이 도는 것이 목표이고**, 그 넷은 “무엇이 깨지면 시연이 안 되는가” 로 고른 것이다.

1.7 6. ④ 회귀 확인 (8단계) — 4차 발표의 핵심 카드

1.7.1 6.1 기준선은 이미 있다

eval/reports/baseline_before.json 커밋 43264e3 · dirty=False · 2판
eval/reports/baseline_before.md
태그 freeze-django-before

지표	기준선
통과	37 ~ 38 / 60 (61.7% ~ 63.3%)
소음대	±1건 (±1.7pp) — 흔들린 건 G-018
▲ 과소평가율	0.0%
▲ 중대 과소	0건
과대평가율	16.7 ~ 19.4% (의도된 편향)
지연 (answered)	p50 12.8s · p95 16.2s

1.7.2 6.2 판정 규칙

```
python eval/harness/run_eval.py --arm A --json eval/reports/baseline_after.json
python scripts/diff_reports.py eval/reports/baseline_before.json eval/reports/baseline_after.json
```

결과	판정
통과 건수 차이 ≤ 1건 2건 이상 ▲ 과소평가율 > 0% 또는 중대 과소 ≥ 1건	같다. 전환이 판정을 바꾸지 않았다 (D-102) 뒤집힌 케이스를 전수로 설명한 뒤 병합한다 통과 건수와 무관하게 되돌린다

마지막 줄이 가장 중요하다. **총계가 좋아져도 과소평가가 생겼으면 회귀다** — 틀리던 건이 거절로 빠져도 일치도는 오르기 때문에, 총계만 보면 방향을 못 읽는다.

1.7.3 6.3 재는 조건을 바꾸지 않는다

temperature=0 이어도 seed 를 주지 않으므로 같은 코드로도 결과가 흔들린다. 그래서 **기준선과 같은 조건**으로만 잰다.

- --arm A · 같은 프로파일(eval) · 같은 top_k(5) · 같은 임계값(0.50)
- 커밋되지 않은 변경이 없는 상태 (freeze_baseline.py 사전 점검이 막는다)
- **pettrriage 를 어느 폴더에서 임포트하는지 확인** — 다른 폴더의 코드를 재면 리포트의 커밋만 이곳을 가리킨다 (2026-08-24 실제로 날 뻔했다)

1.7.4 6.4 ▲ 알려진 오프셋 — G-042 (표기 교정)

기준선을 잰 뒤에 판정 경로가 한 번 바뀌었다. ksr 의 표기 흔들림 교정 (resolve_substance_lenient · 2026-08-26 머지)이 들어왔다.

영향 범위를 **LLM 없이** 확정했다 — 골든셋 60건의 표면형을 교정 함수에 통과시켜 없음 → 이름 으로 바뀌는 자리를 세었다.

G-042 '마카다미아너트' → '마카다미아 너트' ← 유일하다

다른 59건은 교정이 발동하지 않는다. resolve_substance_lenient 는 ⑤**없음일 때만** 개입하므로, 지금 잘 풀리는 것은 건드리지 않는다.

G-042 는 기준선에서 두 판 모두 실패했다 — 등급 2→4 (rule=None llm=4). rule=None 이 원인이다. 물질을 못 잡아 규칙이 등급을 못 냈고 LLM 의 4가 그대로 나갔다. 교정이 붙으면 규칙이 정량 계산을 하고 게이트가 병합하므로 **2로 내려와 정답과 맞을 수 있다.**

8단계에서 보이는 것	읽는 법
G-042 만 달라졌다	전환은 깨끗하다. 이 오프셋이다
G-042 외에 달라진 것이 있다	그것이 전환 탓이다. 전수로 설명한다
G-042 가 그대로 실패한다	교정이 예상대로 안 먹은 것 — 그 자체를 본다

왜 기준선을 다시 박지 않았나. 우리는 이미 어느 케이스가, 왜 움직이는지 안다. 재측정은 그 인과를 숫자 하나로 덮어 버린다 — **+1 이 소음인지 교정인지 구분되지 않는 상태로 되돌아간다.** 알려진 오프셋을 적어 두는 편이 정보가 더 많고 비용이 0 이다.

다만 오프셋이 **둘 이상 쌓이면** 이 방식은 무너진다. 그때는 다시 박는다 (python scripts/freeze_baseline.py --force).

1.7.5 6.5 G-106 을 회귀로 읽지 않는다

기준선 2판에서 **G-106 은 두 판 다 실패했는데 실패 방식이 같았다** (거절이유 근거없음→범위밖 vs 상태 refused→clarify). 통과/실패만 세면 안 보이는 흔들림이므로, 8단계에서 이 건이 또 다르게 실패해도 회귀가 아니다.

1.8 7. CI 게이트

.github/workflows/ci.yml — **모든 브랜치의 push 에서 돈다** (branches: [***]).

잡	설치	검사
lint	ruff (핀)	LINT_PATHS = src · tests · scripts · eval ▲ § 7.1
—	▲ django extra 를 까는 잡이 없다	§ 5.3.1
test	.[api,dev]	pytest -v · 콘솔 스크립트 · 코퍼스 검증
db-deps	.[api,db,dev]	비밀번호 해싱 · 전체 스위트 · 의존성 없으면 기동 중단하는가
rag-deps	.[api,rag,dev]	RAG 경로

더 많이 까는 잡만 초록이면 의존성 누락이다. db-deps 만 통과하고 test 가 죽은 것이 records 라우터가 상시 목록에 잘못 남아 있다는 신호였다 (app/routes/__init__.py 머리말).

1.8.1 7.1 ▲ Django 앱은 린트에도 안 걸려 있다

LINT_PATHS = src tests scripts eval

webapp · accounts · pets · diary · chat 이 빠져 있다. 3711줄이 pytest 밖에 있는 데다 ruff 밖에도 있다 — 지금 diary/views.py 와 webapp/settings.py 에 100쪽을 넘는 줄이 셋 있는데 CI 는 초록이다.

※ ruff 는 글자 수가 아니라 표시 폭으로 잴다 — 한글이 2칸이다. 79자짜리 줄이 101폭이 된다. 한글 주석이 많은 저장소라 이 차이가 계속 물린다.

§ 5.3(pytest-django 도입)과 같은 시점에 함께 켜다. 지금 켜면 팀원 코드에 빨간불이 여럿 뜨고, 그것을 누가 언제 고칠지가 정해져 있지 않다. 검사만 켜고 고칠 사람이 없으면 게이트가 빨간 채로 방치되고, 그때부터 게이트가 아니다.

▲ 실제로 그런 일이 있었다 — 92cd59e(3차 산출물 archive 분리) 이후 make lint 가 계속 빨갓는데 아무도 눈치 채지 못했다. 2026-08-24 에 고쳤다.

1.8.2 7.2 test 잡에 Django 앱을 들이는 방법

§ 5.3 을 하면 test 잡에 Django 앱이 들어온다. pytest-django 를 dev 에 넣으므로 잡을 새로 만들지 않는다.

1.9 8. 무엇을 테스트하지 않는가

적어 두지 않으면 “그건 왜 안 했나” 가 발표에서 나온다.

안 하는 것	왜
LLM 출력의 문장 품질	골든셋의 must_contain·must_cite 로 갈음한다. 사람이 읽는 평가는 4차 범위 밖
근거 검증의 재현율	정답 라벨이 없어 잴 수 없다. 지금 수치는 탐지율이다 (04 § 8)
부하 · 동시 접속	단일 인스턴스 시연 구성이다 (12 § 6)
브라우저 자동화(E2E)	화면 3상태는 통합 테스트로 계약 수준에서 본다 (§ 5.2)
3차 web/ 6장	6단계에서 지을 예정이다 (11 § 6.1)

1.10 9. 선행 관계

- ① 단위 564 ● 지금 돈다 (승계 확인 완료)
- ③ Django 0건 → [1 마이그레이션] → [2 계정] → [3 펫] → [4 리포트 왕복]
- ↑
- 검토 § 2· § 3 결정이 먼저
- ② 통합 63 ← 4·5·6 단계 완료 후
- ④ 회귀 60 ← ②③ 이 끝나고, 5단계(FastAPI DB 라우트 폐기)가 닫힌 뒤

③의 1번(마이그레이션)이 전체의 병목이다. 결정은 끝났고(D-104 · 2026-08-26) 남은 것은 실행이며, 7b 와 같은 작업이다 — FastAPI 의 auth·users·pets·records 라우터를 폐기해야 두 DB 를 하나로 합칠 수 있고, 그 폐기가 통합 63건을 깨뜨린다.

라우터 폐기 → DB 통합 → migrate 가 끝까지 돈다 → 통합 63건 재작성 → 4단계가 닫힌다

↳ 대화 적재를 Django 로 (D-105) · `log\_chat\_turn` 제거 · pet_id 소유권 검증

따로 잡지 않는다. 하나의 작업 덩어리다.

▲ 7b 의 범위가 커졌다. 라우터 폐기 · DB 통합 · 통합 63건 재작성 · Django 앱 테스트 최소 4건 (§ 5.2) · 대화 적재 이관(D-105). 한 사람이 하기에는 크다 — 배분 회의에서 이 덩어리를 어떻게 나눌지 정해야 한다.

1.11 10. 미결

- ☐ § 5.2 1번(마이그레이션) — ● 실행은 끝났다 (pets 0001~0003 · diary 0001 · migrate --check 통과). 남은 것은 그 것을 지키는 테스트다 — 7b
 - ☒ test_api.py 28 → 가를 것이 없다. 전부 /api/ask/api/health 라 그대로 남는다 (§ 4.1.1 · 2026-08-27 실측)
 - ☐ pytest-django 도입 — dev extra + 핀 (§ 5.3)
 - ☐ ▲ CI test 잡에 django extra 추가 (§ 5.3.1) — 지금은 Django 가 CI 밖이다
 - ☐ § 5.2 4번의 추론 서비스 대역 방식 — 대역으로 할지 실제 왕복도 한 건 돌지
 - ☐ ▲ LINT_PATHS 에 Django 앱을 넣을 것인가 (§ 7.1) — 켜면 누가 언제 고치나
 - ☐ 8단계 회귀를 몇 판 둘 것인가 — 기준선이 2판이므로 최소 1판, 여유가 있으면 2판
-

1.12 11. 기여 이력

무엇	누구	원본	반영
계획 통합 · 층 구분 · 회귀 판정 규칙	오한빈	—	2026-08-24
기준선 실측 (§ 6.1)	오한빈	eval/reports/ baseline_before.md	2026-08-24
☐ 통합 63건 재작성 범위	☐	docs/<브랜치>/	☐
수동 UI 테스트 51건 (⑤ · § 2.1) · Pet 정렬 버그 · 병행 라우터 발견	권소라	docs/ksr/...보고서 (UI).xlsx	2026-08-27
인프라 15건 실측 → 10 § 6 의 ☐ 여섯 개를 채움	권소라	docs/ksr/...보고서(최종).xlsx	2026-08-27
⑤ 층 신설 · ordering 수정 · § 4.5 정정	오한빈	흡수	2026-08-27
⑤ 비교에서 결함 셋 추적 → D-108 · D-109 · D-110 · 회귀 테스트 39건	오한빈	흡수	2026-08-27
§ 2.2 테스트케이스 51건 정본 ID 이관 · xlsx 생성기	오한빈	흡수	2026-08-27

1.13 12. 검수 기준

1. § 2 의 수가 pytest 실행 결과와 같다
2. § 5.2 의 넷이 전부 돈다 — 특히 1번(마이그레이션)
3. § 7 의 네 잡이 모두 초록이고, test 잡이 Django 앱을 포함한다
4. § 6.2 판정 규칙으로 8단계 결과가 기록됐다
5. § 10 미결이 비었다